them alongside your regular production code source files, your directories start to get bloated—effectively you end up with two files for every production source file.

A common solution is to have a test/ directory where you place all your test source files. This directory is then placed parallel to the directory containing the code you're developing. For example, for our Roman numeral class, we may have this:

```
roman/
  lib/
    roman.rb
    OTHER FILES
  test/
    test_roman.rb
    OTHER TESTS
   OTHER STUFF
```

This works well as a way of organizing files but leaves you with a small problem: how do you tell Ruby where to find the library files that are being tested? For example, if our TestRoman test code is in a test/ subdirectory, how does Ruby know where to find the roman.rb source file that we are trying to test?

An option that *doesn't* work reliably is to build the path into require statements in the test code and run the tests from the test/ subdirectory:

```
require 'test/unit'
require '../lib/roman'

class TestRoman < Minitest::Test
  # ...
end
```

This doesn't work in general because our roman.rb file may itself require other source files in the library we're writing. The roman.rb file will load them using require (without the leading ../lib/), and because they aren't in Ruby's $LOAD_PATH, they won't be found. Our test just won't run. A second, less immediate problem is that we won't be able to use these same tests to test our classes once installed on a target system, because then they'll be referenced simply using require "roman".

You could do this using require_relative '../lib/roman', which would be more stable and doesn't assume anything about the load path. A better solution is to assume that your Ruby program is packaged according to the conventions we'll be discussing in Chapter 15, Ruby Gems, on page 251. In this arrangement, the top-level lib directory of your application is assumed to be in Ruby's load path by all other components of the application. Your test code would then be as follows:

```
require 'minitest/autorun'
require 'roman'

class TestRoman < Minitest::Test
  # ...
end
```

And you'd run it using this:

```
$ ruby -I path/to/app/lib path/to/app/test/test_roman.rb
```